

PRAXIS

The real-world flight simulator for product managers.
Product Requirements Document · v1.0



Author	Mike Hamata — Founder
Co-author	Claude (Opus 4.7) — Technical PM
Version	1.0 (MVP scope)
Date	2026-04-18
Status	Draft — pending stakeholder review
Working name	PRAXIS (Greek: "practice"). Finalise with trademark check.
Alternatives	Decisionary · Sprintcraft · Keystone · North Star Sim

Table of Contents

1. Executive Summary	13. Feature Specifications (MVP)
2. Problem & Market Opportunity	14. UI / UX Design Principles
3. Target Users & Personas	15. Analytics & Instrumentation
4. Product Vision & Strategy	16. Security, Privacy & Compliance
5. Core Game Mechanics	17. Pricing & Business Model
6. Customer Engagement Model	18. Go-to-Market Strategy
7. Event Engine (Dice Replacement)	19. Cost Projections & Financials
8. Scenario Framework	20. Roadmap & Phasing
9. AI Mentor System	21. Success Metrics & KPIs
10. Multiplayer Architecture	22. Risks & Mitigations
11. Technical Architecture	23. Team & Resourcing
12. Data Model	24. Next Steps

1. Executive Summary

Vision

PRAXIS is the real-world flight simulator for product managers. A browser-based multiplayer simulation where aspiring and working PMs practice decisions under uncertainty — with consequence-driven events, behavioural customer models, and AI-led retrospectives — until product judgment becomes a reflex.

The problem

Product management is learned by doing, but most people can't access the "doing." Books teach theory, Reforge charges \$2,000/year for content consumption, CSPO workshops give 2 days of physical exercises that vanish when the whiteboards are wiped. Nobody offers what pilots and surgeons take for granted: *a simulator where you practice consequential decisions at speed, with instant feedback, as many times as it takes to internalise the skill.*

The solution

PRAXIS turns Zuzi Šochová's Business Value Game and the full CSPO curriculum into a web-native game engine that:

- **Removes all friction** — zero spreadsheets, zero math. Players drag cards, commit decisions, see consequences
- **Replaces random dice with consequence-driven events** — skipped refinement three sprints ago causes scope surprises now
- **Models customers as behavioural state machines**, not happiness integers

- **Generates AI-led retrospectives** after every game with alternate-history replays
- **Supports multiplayer Scrum roles** — PO + SM + Developer co-op with role-specific views
- **Ingests real backlogs** — paste your Linear/Jira, simulate next sprint before running it

Target market (SOM year 1)

200K+ CSPO/CSM certifications/yr	\$400M PM bootcamp market	50K+ Reforge subscribers	\$1B+ Corporate L&D product training
--	---	--	--

MVP scope & timeline

6-week build. Solo mode, one scenario (SaaS / fintech), event engine with 20 cards, 3 customer archetypes, AI post-game retrospective. Targeted launch: **Q3 2026**.

Business model

Freemium with four paid tiers. Year-1 revenue target: **\$200–250K ARR**. Path to \$1M ARR by end of year 2 through Certified Scrum Trainer partnerships.

Why this, why now, why Mike

- AI is commoditising PM *output* (PRD writing, spec generation). The moat shifts to **judgment**. PRAXIS trains judgment.
- CSPO/CSM is a \$1,000-per-cert market with 200K+ attendees annually and no serious digital companion product.
- Mike brings 13 years of operational product experience (Questrade), an active SaaS build (DentinFlow), and is in a live CSPO cohort — unique founder-market fit.

2. Problem & Market Opportunity

The learning gap

Product managers are paid to make decisions under uncertainty. Nothing in existing training produces reps in this exact skill:

Learning modality	What it teaches	What it doesn't
Books / podcasts (Lenny's, Inspired)	Frameworks, principles	Reps. No decisions made. No consequences.
Reforge / Pragmatic	Content consumption	Passive. Exam-style at best.
Case interviews (Exponent)	Interview performance	Transactional. Doesn't build real judgment.

CSPO / CSM workshops	Role clarity, theoretical grounding	Ephemeral. Physical. 2 days then gone.
GoPractice simulator	Growth/analytics PM	Individual-only. No Scrum/PO. No team.
Real on-the-job experience	Everything	Takes 5+ years. Unevenly distributed. No replay.

Market sizing

TAM

\$2B+ global product education and training market (bootcamps, cohorts, enterprise L&D, certifications).

SAM (serviceable)

\$400M — working PMs + aspiring PMs + CSPO/CSM candidates + enterprise Scrum training.

SOM (year 3 target)

\$10M ARR — 3% of SAM via individual subs, CST partnerships, enterprise deals.

Year 1 realistic

\$200–250K ARR — 500 individual subs + 5 CST partners + 2 enterprise pilots.

Competitive positioning (differentiation)

Competitor	Core value	Our differentiation
GoPractice	Growth PM via Amplitude	Scrum/PO focus · multiplayer · zero-friction UX · event causality
Reforge	Content from senior PMs	Practice over content · consequential decisions · lower price point
Pragmatic Institute	Certification	Dynamic scenarios · team play · modern UX · AI feedback
ITERATE (board game)	Physical agile sim	Digital · async · scalable · replay · AI mentor
Capsim	Generic business sim	PM-specific · Scrum-native · modern pricing · self-serve
Forage (EA PM sim)	Free entry-level job sim	Full product · career-long utility · certification path

Our wedge: multiplayer Scrum decision simulation with AI mentor. Nobody owns this intersection.

3. Target Users & Personas

 **Primary: Ascending Amir — Aspiring PM**

28, analyst at a fintech. Studying for CSPO next month. Wants to land first PM role in 6 months. Applying to 20 companies.

Pain: interview prep is memorising frameworks; no way to show judgment. Can't articulate "why I'd order this backlog that way."

 **Primary: Promoted Priya — Working PM**

32, just promoted to Senior PM at a growth-stage SaaS. Manages a 6-person Scrum team. Never formally trained in product.

Pain: team complains about unclear priorities. Stakeholders bypass her to devs. Knows something is broken but can't name it.

Willingness to pay: \$19–49/month personal · urgent because job search

Key JTBD: "Help me demonstrate product judgment in an interview the way Leetcode demonstrates coding."

Willingness to pay: \$49/month personal or team plan via employer

Key JTBD: "Help me rehearse decisions I might make wrong this week."

🎯 Secondary: Training Tara — CST / Educator

48, Certified Scrum Trainer. Runs 2-day CSPO cohorts monthly. Uses Mural + physical exercises.

Pain: students forget the game after class. No continuation. Hard to differentiate from other CSTs.

Willingness to pay: revenue share per student (est. \$20 per cohort participant)

Key JTBD: "Give me a digital companion my students use during and after my workshop."

🎯 Secondary: Enterprise Eva — L&D Lead

45, leads People Ops at a 2,000-person company. Recently mandated Scrum for R&D. Needs to train 40 managers.

Pain: live workshops cost \$80K+; retention poor; no way to measure learning outcomes.

Willingness to pay: \$10K–50K/year enterprise seats with reporting

Key JTBD: "Certify 40 managers in product skills with measurable outcomes, for under \$50K."

4. Product Vision & Strategy

North Star

Every active user makes at least 5 consequential product decisions per week, with feedback loops under 60 seconds — for under \$50/month.

Positioning statement (Moore template)

For aspiring and working product managers who need reps in decision-making under uncertainty,

Who can't afford \$2K/yr Reforge or \$80K/yr live workshops and don't learn from books alone,

PRAXIS is a multiplayer product-management simulation

That lets them practice real Scrum/PO decisions with consequence-driven events and AI-led retrospectives,

Unlike GoPractice (analytics-only), Reforge (content-only), or physical workshops (ephemeral),

Our product turns every decision into an immediately-consequential moment — and every game into a portfolio artifact you can show a hiring manager.

Three-year vision

Year	Positioning	Scope	ARR target
Year 1 (2026)	"Flight simulator for PM interviews"	Solo, 10 scenarios, AI mentor	\$250K

Year 2 (2027)	"The Scrum training platform for CSPO candidates"	Multiplayer, 40 scenarios, CST partnerships	\$1M
Year 3 (2028)	"The standard in product judgment training"	Enterprise, credentials, MBA curriculum integration	\$3M+

Strategic bets

- **Bet 1: Practice beats content.** The Reforge/Pragmatic era sells content. The next era sells reps. We bet the market is ready.
- **Bet 2: CST partnership is the fastest distribution.** Hundreds of CSTs train ~20K students/year. We license our engine to them, rev-share, reach students at the moment of peak learning demand.
- **Bet 3: AI mentor is a moat, not a feature.** Retrospective quality improves with every game played. Claude Opus 4.7 for depth; Haiku 4.5 for in-game events. Data compounds.
- **Bet 4: Zero-friction UX is the unfair advantage.** Every competitor has friction. We remove all of it. Drag, commit, learn.

5. Core Game Mechanics

Game structure

A **Scenario** = 6–24 iterations (sprints) based on mode. Each iteration has 5 phases:

1. **Discovery** — review backlog, customer signals, prior retro notes
2. **Planning** — order iteration backlog, set Sprint Goal (text), commit investments
3. **Commitment** — lock the plan (like rolling the dice, but without dice)
4. **Execution** — events unfold; player has 1–2 response moments
5. **Review + Retro** — see results, customer reactions, AI feedback

Modes

Mode	Iterations	Focus	Time
Tutorial	3	Mechanics, ordering, release	20 min
Quick	6	Classic CSPO-style game	45 min
Quarter	12	Adds hiring, budget, OKRs	2 hrs
Year	24	Scaling, PMF crisis, competition	4 hrs (or async)
Interview	3	Single high-pressure scenario, scored	30 min

What the player manipulates (zero-friction UI)

- **Drag to order** — the iteration backlog is a list; player drags items into order

- **Drop the release card** — insert the release card anywhere in the ordered list; everything above it releases if done
- **Click to commit** — one button, locks the order, triggers execution
- **Respond to events** — when an event fires, a modal offers 2–3 choices
- **Write a Sprint Goal** — one sentence; graded by AI for clarity & outcome-orientation
- **Accept / decline customer asks** — each incoming ask shows its effort cost; declining has relationship consequences

All math (velocity calculation, happiness deltas, interest accrual, release checks) is hidden. The player sees *consequences as stories*, not numbers.

Commitment cap & story cost (preserves the dice's "you can only do so much" lesson)

Removing the dice must not remove the lesson that *velocity is a hard constraint*. PRAXIS handles this with three mechanics:

Mechanic 1 — Every story carries an explicit effort cost

- Effort (points) is visible as a badge on each card
- Cards are visually scaled to their effort (5-pt card is physically larger than 2-pt card)
- Some items carry "effort: 5?" — meaning true cost is only revealed during execution (teaches refinement risk)
- Release card always costs a fixed amount (3 pt by default, reducible via tech investment)

Mechanic 2 — Capacity range indicator (not a single number)

As the player drags cards into the iteration backlog, a capacity bar fills with zones:

Zone	Visual	Meaning
0 → lower bound (e.g. 12)	 Green	Safe — guaranteed to complete
Lower → expected (e.g. 15)	 Yellow	Realistic — completes in most sprints
Expected → upper (e.g. 18)	 Orange	Stretch — only completes on a good sprint
Above upper bound	 Red + wiggle	Over-committed — tail items almost certainly drop

Critical design decision: PRAXIS does *not* block the player from over-committing. The visual warning is clear but the decision is theirs. Over-commitment is a muscle you unlearn through pain, not prevention. The execution phase shows what actually happened, and any items below the cutoff line fall back to the product backlog with a "didn't make it" animation.

Mechanic 3 — Capacity range shifts based on state (replaces random dice)

The range itself moves deterministically based on team state and prior decisions. No random roll.

State condition	Effect on capacity range
Fresh team, refined backlog	Tightens: 14–16 pts (high predictability)
Engineer on vacation / sick	Shifts down: 10–14 pts

New hire ramping up	Widens: 10–17 pts (more variance)
Invested in DoD + automated tests	Tightens over time (variance reduces)
Scope surprise event mid-sprint	Narrows mid-execution: 12–14 pts
Team burnt out from context-switching	Shifts down AND widens: 9–15 pts

Customer asks as costed commitments (the "cost per ask" lesson)

Every customer ask that lands in the inbox includes its effort cost upfront. The player chooses:

- **Add to backlog** — locks the cost into future capacity planning
- **Decline politely** — costs relationship points with that customer
- **Negotiate** (v2) — smaller scope variant for less effort

This is the mechanic that teaches "*saying yes to everything fills your backlog with items that consume velocity.*" The PO skill of ordering + refusing is built here, not in the planning board alone.

Execution resolution (what happens after commit)

1. Player commits an ordered iteration backlog (may be over capacity)
2. Events fire based on behaviour-weighted deck
3. Actual capacity resolves deterministically from state (no random dice)
4. Items execute top-to-bottom in the committed order
5. Items completing within capacity go to Done
6. Items below the cutoff line fall back to Product Backlog with visual feedback
7. If the release card is below the cutoff, nothing releases this sprint (expensive but instructive)

Teaching goal preserved: players learn that velocity is a real constraint, that each story has a non-negotiable cost, and that over-committing produces worse outcomes than planning to the lower bound. This is exactly the dice's lesson — delivered through visible causality instead of random rolls.

Hidden state exposed through signals

Hidden state	What player sees
Customer happiness score	Message from customer: " <i>Priya: 'I'm starting to wonder if you heard my last ask'</i> "
Velocity modifier	Team vibe signal: " <i>The team is in flow this sprint</i> " or " <i>Two devs are out sick</i> "
Tech debt accumulation	Warning banner: " <i>Item X will take 50% longer — hidden complexity</i> "
Stakeholder trust	Exec reaction: " <i>The CFO nods thoughtfully</i> " vs " <i>The CFO leaves the review early</i> "

Scoring (end-game)

Not a single dollar number. A multi-axis radar chart:

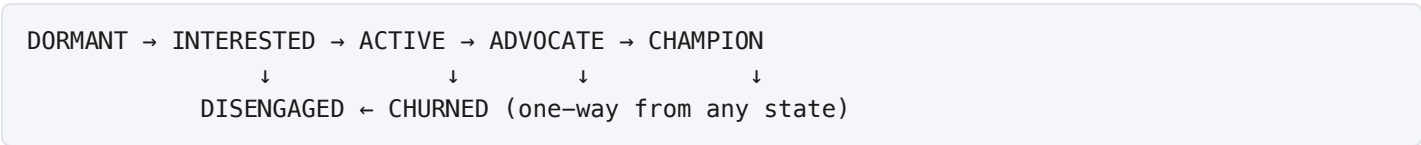
- **Value delivered** — revenue + strategic wins
- **Customer loyalty** — happiness retention, referrals

- **Team health** — morale, sustainable pace, no burnout
- **Stakeholder trust** — alignment across functions
- **Product integrity** — tech debt managed, DoD respected
- **Personal growth** — new techniques unlocked, mistakes avoided vs prior games

6. Customer Engagement Model

Real customers don't have a single "happiness" number. They have **engagement states** that transition based on product behaviour and triggers.

Customer states (state machine)



Customer archetypes

Archetype	Feedback frequency	Patience	LTV	Voice
Innovator	High — unsolicited ideas	Very high (tolerates breakage)	Low-medium	Quiet
Mainstream	Medium — responds when asked	Medium	High	Average
Enterprise	Low — quarterly reviews	Very low for SLAs	Very high	Loud when unhappy
Skeptic	Medium — mostly negative	Low	Medium	Loud
Power user	Very high — detailed	Medium	Medium	Influential to peers
Lurker	Almost none	Unknown	Unknown	Silent

What causes engagement transitions (teachable causality)

Trigger	Effect
Release addresses customer's explicit ask	+1 engagement state; 20% chance to unlock referral
Release for someone else, customer ignored	−0 first time, −1 if repeated 2× in row
Bug/regression in a feature they used	−1; possible vocal complaint event
Response to their feedback within 1 sprint	+1; unlocks "Power user" path
Acknowledgement without delivery	+0 (short-term); −1 (if repeated 3×)
Exciter feature from Kano	+2; 40% advocate promotion
Competitor launches before you	−1 if customer is Innovator type (not others)

Why some customers become more engaged later (teaching moment)

The game teaches this explicitly through scenario narratives:

- **Activation moment:** a release finally solves a pain the customer didn't articulate but was feeling. Triggers engagement jump. The lesson: customers often can't name what they need; observation beats surveys.
- **Social proof:** another customer of the same archetype becomes vocal; this one wakes up to try. Lesson: community signals matter.
- **Credibility accumulation:** after 3 consecutive on-time commitments, a skeptic becomes active. Lesson: trust is earned in drops, lost in buckets.
- **Role change:** the customer gets promoted; now they need different outcomes. Lesson: stakeholders are moving targets.
- **External event:** regulation change makes your product suddenly essential. Lesson: context shifts value.

Feedback mechanisms in-game (what the player sees)

- **Inbox messages** — customers send feedback, bug reports, feature requests; frequency varies by archetype
- **Quiet signals** — dormant customers show small activity pulses (or don't)
- **Reviews** — occasional public reviews that affect acquisition of new customers
- **Referrals** — advocate-level customers bring in new ones at random
- **Churn warnings** — only visible if the player looks at the right signal (teaches attentiveness)

7. Event Engine (Dice Replacement)

The original Business Value Game uses a d6 for velocity variance. PRAXIS replaces this with a **weighted event deck** where probabilities shift based on the player's prior choices. Every event is a teaching moment because it's caused.

Event categories & base weights

Category	Base weight	Example events
Team health	15%	Engineer sick · New hire ramping · Team in flow state · Burnout warning
Tech surprises	20%	Hidden debt on item X · API vendor outage · Flaky test eats 2 days · Perf regression
Dependencies	15%	Legal review delayed · Platform team migration · Design review bottleneck
Scope surprises	15%	Item X is 2x effort · Hidden acceptance criteria · Integration doesn't work
Stakeholder events	15%	Exec demands feature · Customer escalation · Sales lost deal · Analyst call
Positive events	10%	Pair programming breakthrough · Earlier investment pays off · Press mention
External shocks	10%	Competitor launch · Regulation change · Macro event · Security advisory

How PO behaviour shifts probabilities

Prior behaviour	Probability shift
Skipped refinement 2+ times	Scope surprises: +15% · Estimate accuracy: –30%
Invested in DoD	Tech surprises: –20% · Positive events: +5%
Invested in tests / automation	Bug events: –40% · Velocity variance: –25%
Changed priorities mid-sprint 2+ times	Team health –1 permanent · Morale events: +20%
Didn't release for 2 iterations	Stakeholder events: +30% · Customer churn warnings: +25%
Didn't talk to users (no discovery activity)	"Wrong feature shipped" event: +40%

Event response mechanics

When an event fires, a modal offers 2–3 choices. Each choice has a visible and hidden consequence. Examples:

Event: Exec demands a scope addition mid-sprint

Options:

- **Accept the scope** — exec trust +2, team morale –2, tech debt +1, current Sprint Goal probability of completion –30%
- **Defer to backlog refinement** — exec trust –1 (short term), team morale 0, Sprint Goal preserved
- **Trade: accept scope, drop equivalent existing item** — exec trust +1, team morale 0, no tech debt, current sprint goal mutated

Deck composition by scenario

Each scenario has its own event deck (20–50 cards). The deck mixes universal PM events (the above) with domain-specific ones (e.g., "Regulator asks for audit trail" in a fintech scenario; "HIPAA alert" in a health scenario).

7B. Technology & Platform Economics

Every sprint runs on top of code, infrastructure, tooling, and team practices that either **compound advantage** (investments pay forward) or **compound cost** (debt accrues silently). PRAXIS models this as a first-class game mechanic because it's the hardest real-PM decision: invest in the platform now and ship fewer features this sprint — or postpone and pay growing interest.

Principle: The dice in the original BVG teaches velocity variance. The tech backlog teaches the compounding nature of platform investment. Both must be modelled explicitly for the game to be complete.

Six hidden state variables tracked per game

Variable	Start state	How it moves
Release cost	3 pt per release	↓ with CI/CD, deploy automation investment · ↑ silently with debt accumulation
Capacity baseline	15 pt/sprint	↑ with framework upgrades, team training · ↓ with burnout, debt
Capacity variance	±3	Tightens with test coverage, DoD discipline · Widens with new hires, skipped refinement
Tech debt score	0–100 scale	Rises from shortcuts, skipped investment · Falls from refactors
Reliability	High	Falls with cut corners · Rises with monitoring, test coverage
Cycle time	Baseline	Shortens with automation · Lengthens with accumulated debt

Tech Backlog (separate from Product Backlog)

A dedicated section of the backlog where the player can invest effort in platform work. Benefits apply to *future* iterations, not the current sprint.

Tech item (example)	Effort	Benefit (starts next iteration, permanent)
CI/CD Pipeline	5 pt	Release cost drops from 3 → 1 pt permanently
Automated test suite	4 pt	Bug event probability –50%; variance tightens by 1
Observability platform	3 pt	"Hidden complexity" events eliminated
Refactor core module	6 pt	Tech debt –30; future estimates 20% more accurate
Dev team training (bundle)	3 × 1 pt	+1 capacity baseline for rest of game (when all 3 done)
Framework upgrade (bundle)	3 × 2 pt	+2 capacity baseline for rest of game (when all 3 done)
Knowledge documentation	2 pt	Eliminates "key person leaves" risk; new-hire ramp halved
Security hardening	4 pt	Removes security events from the deck
Design system	5 pt	UX-related stories cost 30% less going forward
Data platform	6 pt	Unlocks analytics-based scenarios; +1 per-sprint bonus to discovery

Scenarios define their own tech backlog, scaled to their domain (a fintech scenario would include compliance tooling; a consumer scenario would include marketing infrastructure).

Debt accumulation — the cost of ignoring

Tech debt accrues silently from:

- Shipping items with incomplete Definition of Done (+5 debt per item)
- Declining tech investment when the team flags it (+3 debt per refusal)
- Every 3 iterations with zero tech investment (+10 debt automatic)

- Responding to an event with the "shortcut" option (+2 to +8 debt depending on severity)

Debt thresholds — compounding consequences

Debt level	Consequence
≥ 30	Release card silently grows from 3 → 4 pt
≥ 40	Capacity variance widens by +1 (more unpredictable sprints)
≥ 50	Customer-facing incident events enter the deck
≥ 60	Release cost: 4 → 5 pt
≥ 70	Capacity baseline drops by 2 (fewer points per sprint)
≥ 80	Release cost: 5 → 6 pt; team morale events trigger; engineers quit
≥ 90	"Critical outage" events force-fire; may end the game early in some scenarios

Thresholds are *permanent until reversed* — reducing debt back below 30 doesn't automatically shrink release cost. Players must explicitly invest in the right kind of refactor to reverse the damage. This models reality: once a platform decays, undoing the rot takes more effort than preventing it.

Making it teachable — narrative over numbers

Following the zero-friction principle, tech state is **never shown as a raw number**. It's exposed through in-game narrative signals:

Hidden state	What player sees
Debt rising past 30	"The test suite is flaking. A dev mentions the backend is getting hard to change."
Release cost climbing	"Last deploy took 2 days. The team spent more time debugging than shipping."
Capacity variance widening	"The sprints feel inconsistent. Some weeks the team crushes it; other weeks nothing ships."
Capacity baseline dropping	"Sprints feel slower. Nobody can put their finger on why."
Post-investment payoff	"CI/CD is live. Deploys are one-click. Team visibly happier. Velocity is trending up."
Refactor payoff	"The core module refactor is in. Estimates feel more accurate; scope surprises have dropped."
Silent decay	"A senior engineer submits notice. They cited 'tech debt' as the reason."

Tech investment payoff moments (emotional design)

Each tech investment triggers a **visible payoff moment** the iteration after it lands — same celebration as a customer release. This is a deliberate design choice: real orgs under-celebrate platform wins; PRAXIS over-corrects so players *feel* why infrastructure matters.

The core PM teaching moment this creates

Every scenario forces the decision: **invest now** (sacrificing customer work this sprint) or **postpone** (paying growing interest later). The end-game retrospective explicitly grades tech decisions:

"You declined 3 tech investments in favour of features. Release cost rose from 3 to 5 pt. Your final sprint cost you 2 releases you would otherwise have made. A senior PM would have paid down CI/CD in iteration 2, worth +\$4,200 in compounded value."

Integration with the event engine (Section 7)

Tech state directly feeds the event deck weights from Section 7. For example:

- Low reliability → higher "outage" event weight
- High debt → higher "scope surprise" event weight
- High test coverage → lower "bug" event weight
- Wide capacity variance → more variable iteration outcomes

This means **tech decisions and customer decisions are not independent systems** — they both feed the same event probability model. Neglect either side and events pile up. Balance both and events stay manageable.

8. Scenario Framework

What a Scenario is

A self-contained game world. Includes: product vision, personas (3–8), backlog (20–100 items with effort + dependencies), event deck (20–50 cards), win/lose conditions, narrative beats (tied to iteration numbers).

Scenario library roadmap (30 scenarios across 4 phases)

The scenario library is the primary content engine. Launch with 10, expand to 30+ across Year 1. Scenarios are grouped by the skill cluster they target.

Phase 1 — MVP (10 scenarios · core mechanics + foundational skills)

#	Title	Domain	Level	Teaches
01	The First Sprint	SaaS generic	Tutorial	Ordering, release mechanics, customer happiness
02	The MVP Race	Consumer app	Beginner	Narrowest wedge, customer discovery
03	Churn Cliff	B2B SaaS	Beginner	Retention, customer engagement model
04	The Regulated Product	Fintech	Intermediate	Compliance constraints, platform economics
05	Scope Creep City	Enterprise	Intermediate	Saying no, Sprint Goal defense
06	The Platform Pivot	SaaS	Intermediate	Strategic change, tech debt payoff
07	Two-Sided Marketplace	Marketplace	Advanced	Multi-stakeholder optimization

08	The Post-Launch	Consumer	Advanced	PMF detection, growth vs retention
09	Incident Response Week	B2B SaaS	Advanced	Crisis management, blameless postmortems
10	The Interview Scenario	Generic	Expert	30-min scored scenario for interview prep

Phase 2 — PM ↔ Developer Dynamics (10 scenarios · the highest-value missing content)

#	Title	Level	Teaches
11	The Hero Engineer	Intermediate	Rebuild temptation · when to respect eng judgment vs push back
12	The Estimate Wars	Intermediate	Calibration conversation · asking "what would make this smaller?"
13	The Tech Lead Disagreement	Advanced	Public disagreement navigation · never undermine, never capitulate publicly
14	The Burning Engineer	Advanced	Recognizing burnout as leading indicator · team health intervention
15	The Goodbye Sprint	Intermediate	Knowledge transfer economics · saying no to one more feature
16	The New Hire Ramp	Beginner	Ramp-up investment · managing impatient stakeholders
17	The Design vs Eng Standoff	Intermediate	Mediation · surfacing value trade-offs without picking sides
18	The Tech Debt Negotiation	Intermediate	Quantifying "if we don't do this, next sprint costs X more"
19	The Dev Who Won't Talk to Users	Beginner	Building customer empathy as eng skill without forcing
20	The "Can't We Just" Moment	Beginner	Humility · responding to "this is actually complex" with curiosity

Phase 3 — Stakeholder Management & Strategic Judgment (10 scenarios)

#	Title	Level	Teaches
21	The Shadow PM	Advanced	Boundary defense · correcting patterns without breaking relationships
22	The Sales Promise	Advanced	Revenue-vs-roadmap integrity · trade-off conversations with sales
23	The 10x Customer	Advanced	The whale trap · avoiding becoming a custom services org
24	The Pivot Call	Advanced	Leading team through strategic uncertainty
25	The Cross-Team Blocker	Intermediate	Influence without authority · trading favors, aligning bosses
26	The Roadmap Review	Intermediate	Executive communication · pre-baked answers to "why not X"
27	The Strategic No	Advanced	Saying no to features that would work but don't fit vision
28	The Sunset Decision	Expert	End-of-life management · customer, team, sales communication
29	The Pricing Change	Expert	Monetization experiments · grandfathering, positioning, testing
30	The Ethics Call	Expert	Values-based decisions when "legally fine" diverges from "right thing"

Phase 4 — Operational Reality & Career (10 scenarios · Year 2 content)

#	Title	Teaches
31	The Measurement Problem	Instrumentation discipline · backfilling from other signals
32	The Inconclusive A/B	Statistical rigor vs shipping pressure
33	The Public Churn	Reputation management · responding to public customer complaints
34	The Contradictory Feedback	Segment thinking · not all customer voices are equal
35	The Silent Majority	Survivor bias in feedback · talking to silent users
36	The Metric That Lied	Goodhart's Law · when hitting targets hurts the goal
37	The Discovery Disconnect	Navigating user research vs leadership intuition
38	The Reorg	Change management from the receiving side
39	The First 90 Days	Onboarding playbook · building trust before changing things
40	The No-Win Meeting	Facilitation under fire · alignment as a floor not ceiling

Bonus: Named case studies (community-contributed, Year 2+)

High-fidelity recreations of famous product decisions: the Slack pivot (from Glitch), the Figma browser bet, the Nokia smartphone miss, the Quibi launch, the Instagram sale. Each uses real public context with fictionalized names. Teaches judgment by replay.

Bonus: Industry packs (community-authored, Year 2+)

- **Fintech pack** — regulated product, clearing/settlement scenarios, KYC/AML decisions
- **Healthtech pack** — HIPAA/PHIPA constraints, clinical workflow integration
- **Marketplace pack** — supply/demand balancing, trust & safety events
- **AI product pack** — hallucination risk, model cost economics, evaluation design
- **Platform/API pack** — developer experience, versioning, deprecation

Scenario authoring (for v2+)

YAML/JSON-based scenario format so CSTs and community members can author their own. Schema:

```
scenario:
  id: "churn-cliff"
  title: "Churn Cliff"
  domain: "B2B SaaS"
  iterations: 6
  product_vision: "... "
  personas:
    - id: "priya"
      archetype: "Enterprise"
      starting_state: "ACTIVE"
      ltv: 50000
  backlog:
    - id: "feature-a"
      title: "... "
      effort: 5
```



```
value: 2500
satisfies: ["priya"]
tech_debt_if_skipped: true
event_deck:
- id: "churn-warning"
  weight_base: 0.15
  weight_modifiers:
    no_refinement: 1.5
  trigger_after_iter: 2
```

8B. PM Behavior Patterns & Interaction Nudge System

PRAXIS teaches product management through scenarios, but scenarios alone are not enough. The deepest value comes from recognizing **behavioral patterns** that play out across many scenarios — the same mistakes in different costumes. A player may win a scenario but lose the career-long game if they don't see the pattern. This section defines the pattern-recognition system that turns PRAXIS from a knowledge-delivery tool into a behavior-change tool.

Design principle: Most PM training teaches frameworks. PRAXIS teaches reflexes. Reflexes are built by catching yourself in the moment, repeatedly, until the right move becomes automatic. The nudge system is how we produce that catch.

PM ↔ Developer interaction patterns (reference model)

Each pattern below has a "healthy" and an "anti" behavior. The game observes the player's choices and tracks both.

Interaction context	Healthy PM behavior	Anti-pattern (tracked)
Backlog refinement	Tell the "why" and "who"; answer questions; stay available; let devs propose "how"	Specifying implementation; drifting into technical solutioning; writing pseudo-code in tickets
Daily Scrum	Usually doesn't attend. If invited, observes silently.	Using it as a status report; adding scope mid-sprint; interrogating individuals
Estimation	Asks "what would make this smaller?"; surfaces the business value of a smaller scope	Challenging estimates; suggesting corners to cut; anchoring to an external deadline
Sprint Planning	Presents ordered backlog; explains priority logic; answers clarifying questions	Dictating scope; overriding team commitment; negotiating individual items with individual devs
Sprint Review	Frames why the work matters; invites stakeholder feedback; updates backlog from it	Treating it as a demo gate; surprising stakeholders; blaming team for incomplete items
Retrospective	Attends only if invited; shows up as team member not boss	Facilitating (SM's job); using it to pressure team; defending own decisions
1:1 with Tech Lead	Weekly cadence; surfaces blockers early; keeps them informed of roadmap shifts	Treating as status meeting; skipping when busy; going around them to other devs

Incident Response	Owens customer communication; lets engineering own technical diagnosis	Jumping into technical debate; "what if we just..." in the war room; blaming during the incident
Tech Debt Discussion	Asks "what does it cost us if we don't?"; listens for quantifiable answer	Dismissing as "not now"; accepting every refactor proposal without scoping
Team Health	Notifies burnout signals; celebrates wins; protects the team from thrash	Cheerleading when something's really wrong; performative concern; ignoring signals
Stakeholder Pressure	Translates pressure into trade-offs; surfaces to team transparently	Absorbing pressure silently and passing as urgency; promising without consultation
Technical Disagreement	Acknowledges publicly, resolves privately, never undermines	Overruling in-channel; capitulating publicly; dragging into long thread

Pattern tracking layer (telemetry model)

Every player action is classified against the pattern table and logged. Over many games, patterns emerge as signals, not single incidents.

Pattern tag	Triggered by	Surfaced when
<code>specifies_how</code>	Sprint Goal or event response contains implementation detail (AI-classified)	3+ times in 5 games → retro flag
<code>mid_sprint_scope_add</code>	Player accepts scope during execution phase	2+ times in 3 games → immediate in-game nudge
<code>overrides_tech_lead</code>	Player picks the "override" option in a technical disagreement event	Any single instance → flagged in retro
<code>skips_refinement</code>	Player declines the refinement investment option	2+ consecutive sprints → scope-surprise event probability rises
<code>ignores_tech_debt</code>	Player ignores tech signals for 3+ iterations	Debt crosses threshold → narrative signal fires
<code>yes_to_everything</code>	Accepts all customer asks in 2+ consecutive iterations	Backlog balloons → capacity warnings
<code>no_user_contact</code>	No customer inbox clicks for 2+ iterations	Discovery drought → "wrong feature shipped" event weight rises
<code>changes_priority_mid_sprint</code>	Player re-orders backlog after commit	Any instance → team morale -1; pattern in retro
<code>owns_the_demo</code>	Player chooses "I'll present" over "Let the devs present" in Sprint Review events	Pattern in retro: "consider letting the team own the showcase"
<code>cheerleader_mode</code>	Player dismisses team health warnings 2+ times	Leading indicator of burnout event within 2 iterations
<code>absorbs_pressure</code>	Player accepts exec scope add without team discussion	Team trust -1; pattern in retro

skips_retro	Player declines retro investment or doesn't set improvement goal	Process debt accrues; retro flag at end of game
-------------	--	---

Real-time nudge system (in-game coaching)

When a player is about to take an action that matches a known anti-pattern, PRAXIS surfaces a **subtle nudge** — not a blocker. The player still makes the choice, but now with awareness.

Nudge delivery rules

- **Never block.** The player can always proceed with the anti-pattern choice. Consequences teach; popups preach.
- **Never interrupt mid-drag.** Nudges appear on commit, not during planning.
- **Time-box to once per pattern per game.** If a player sees the "specifies_how" nudge once, they don't see it again that game — prevents nagging.
- **Tone: curious, not corrective.** "Notice this pattern?" not "This is wrong."
- **Opt-out available.** "Expert mode" disables in-game nudges; all tracking still happens for the retrospective.

Example nudges

Trigger	Nudge text
Sprint Goal contains "build a" or "implement"	"Heads up — your Sprint Goal reads like a 'what' more than a 'why'. Outcomes travel further than outputs. Proceed anyway?"
Player accepts mid-sprint scope add	"You've taken a mid-sprint scope change. This is the 3rd this game. The team notices. Continue?"
Player overrules tech lead in disagreement event	"You're overriding the tech lead publicly. A senior PM might have taken this offline. Are you sure?"
Player hasn't opened inbox in 2 iterations	"Your customers haven't heard from you in 2 sprints. Discovery signal is fading."
Player declines tech investment 3rd time	"Third tech investment declined. The team is quietly escalating debt. Continue?"

Cross-game pattern recognition (the moat)

Single games teach scenarios. Cross-game pattern detection teaches *you as a PM*.

Player profile (private, visible to the player only)

After 5+ games, the player sees their own pattern dashboard:

- **Strengths** — patterns they consistently do well (e.g., "Strong at customer coverage — 9/10 iterations with active customer engagement")
- **Edges** — patterns they consistently struggle with (e.g., "Tends to accept scope mid-sprint — 11 instances across 8 games")

- **Trends** — patterns improving or declining over time (e.g., "Tech debt decisions improving — declined 3 investments in early games, 0 in last 3")
- **Unlocks** — mastery badges earned for 10+ consistent healthy patterns

AI retrospective integration (links to Section 9)

The end-of-game retrospective uses the pattern tracker as input. Instead of generic feedback, the AI mentor surfaces specific pattern evidence:

"Across this game and your previous 4, you've overruled your tech lead 3 times — twice publicly. Each time, the decision worked out in the short term, but team trust eroded. A senior PM would have taken the first disagreement offline and established the pattern that technical decisions live with engineering. Here's what that would have looked like in this game's Iteration 3..."

Cohort-level insights (trainer feature, Phase 2)

CSTs and L&D leads get a cohort dashboard showing aggregate patterns across their trainees (anonymised in aggregate, attributable only to the trainer). Answers questions like:

- What's the most common anti-pattern in this cohort?
- Which trainees are showing rapid behavior change?
- Which scenarios triggered the most learning moments?

This turns PRAXIS from individual training into measurable organizational learning — key for enterprise sales.

Integration with the core engine

- Pattern classification runs server-side on decision events
- Claude Haiku 4.5 classifies text inputs (Sprint Goals, event responses) for implicit patterns
- Rules-based classification for structural patterns (mid-sprint changes, inbox ignores)
- Patterns stored in PostgreSQL linked to Game + User; aggregated in UserProgress
- Cost per game: ~\$0.04 (Haiku classification) + ~\$0.12 (Opus retro) = ~\$0.16 all-in

Strategic implication: the pattern library is proprietary content that compounds. Each player's games train the system on what "good" and "anti" look like across thousands of decision points. Competitors starting later can't easily replicate the behavioral training data.

9. AI Mentor System

Two distinct AI surfaces

Surface	Model	Latency req.	Cost target
In-game event generation (dynamic events)	Claude Haiku 4.5	<2s	<\$0.02 per event
Sprint Goal clarity grading	Claude Haiku 4.5	<3s	<\$0.01 per grade

Post-game retrospective	Claude Opus 4.7 (1M ctx)	<30s	<\$0.30 per retro
Stakeholder roleplay (v2)	Claude Sonnet 4.6	<4s	<\$0.05 per turn

Post-game retrospective — structure

AI-generated retrospective template (per game)

1. **Headline** — one sentence that captures this game's defining decision

2. **3 strengths** — with specific moments timestamped

3. **3 growth edges** — with specific alternative decisions

4. **Alternate history replay** — "Here's what would have happened if, in iteration 3, you had..."

5. **Technique unlocked** — one framework the player demonstrated readiness for (Kano, WSJF, ICE, Cost of Delay, etc.)

6. **Recommended next scenario** — based on growth edges

Prompt scaffolding (Opus 4.7)

The retrospective prompt includes: full game log (decisions, events, outcomes), scenario metadata, player's prior games (if any), PM principles library (Cagan, Torres, Patton, Šochová). Output validated against JSON schema for consistent UI rendering.

Prompt caching strategy

Scenario definitions + PM principles library + retrospective rubric are static per game — **cache them with 5-minute TTL**. Expected cache hit rate: 70%+, reducing average retrospective cost from \$0.30 to ~\$0.12.

Content safety

- Retrospective tone is constructive, not shaming. Prompt explicitly forbids ridicule.
- No personal data in prompts (use game state IDs, not real names).
- All AI output labelled; players know what's AI-generated.

10. Multiplayer Architecture

Modes

Mode	Players	Synchronicity	Priority
Solo	1	N/A	MVP
3-role Co-op (PO/SM/Dev)	3	Real-time	Phase 2
Tournament	Unlimited, same scenario	Async	Phase 2
Async campaign	3	1 iteration/day	Phase 3
Classroom mode	10–50 per cohort	Facilitator-paced	Phase 3

Role-specific views (Co-op)

Role	Can see	Can do
PO	Full backlog, customer signals, stakeholder messages, budget	Order backlog, set Sprint Goal, respond to stakeholder events
SM	Team health, impediments, facilitation opportunities	Remove impediments, call retros, coach team decisions
Dev	Backlog + technical detail, DoD, code health	Commit to sprint, flag scope surprises, estimate effort

Technical layer

- WebSocket channels per game (Socket.IO or Pusher)
- Authoritative server state in Postgres; Redis for ephemeral game state
- Optimistic UI for decision drag-drop; server reconciles
- Spectator mode: trainers can observe any game they have access to

11. Technical Architecture

Stack

Frontend:

Next.js 14 (App Router, Server Components, Suspense streaming)
React 18 + TypeScript
dnd-kit (drag-drop for backlog ordering)
Framer Motion (event reveals, release animations)
Tailwind CSS + shadcn/ui
Zustand (client state)

Backend:

Next.js API Routes + tRPC for type-safe APIs
Node.js 20 runtime
Prisma ORM
PostgreSQL 16 (primary data store)
Redis (Upstash) – ephemeral game state, pub/sub
Socket.IO or Pusher for WebSocket multiplayer

AI/ML:

Anthropic SDK (claude-opus-4-7, claude-haiku-4-5, claude-sonnet-4-6)
Prompt caching with 5-minute TTL
Structured output via JSON schema

Auth:

Clerk (MVP) or NextAuth.js (Phase 2)

Payments:

Stripe (subscriptions, metered billing for enterprise)

Observability:

PostHog (product analytics – Mike's existing stack)
Sentry (error tracking)
Vercel Analytics (Core Web Vitals)

Hosting:

Vercel (frontend + API routes)
Railway or Supabase (Postgres)
Upstash (Redis)

CI/CD:

GitHub Actions
Preview deploys on Vercel per PR
Vitest for unit, Playwright for E2E

Systems diagram (narrative)

Player's browser runs Next.js app. Game engine state lives in Postgres (persistent) + Redis (hot state during play). Decisions mutate state via tRPC. Event engine runs server-side, consults Claude Haiku for dynamic narration, writes results back. Post-game, Claude Opus generates retrospective (background job). PostHog ingests every decision event for analytics.

Game engine design (critical)

- **Pure deterministic core** — given state + action, always produces same result. Enables replay, debugging, and scenario testing.
- **Event log** — every decision and outcome stored as append-only log. Enables replay and alternate-history simulation.
- **Deterministic randomness** — seed-based PRNG per game; same seed + decisions = same game. Enables "replay with different strategy."
- **Engine is a library**, not a service — can run in browser for tutorial, on server for real games.

12. Data Model

```
// Core Prisma schema – simplified for PRD

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String?
  tier         Tier       @default(FREE)
  createdAt   DateTime  @default(now())
  games       Game[]
  progress    UserProgress?
  cohortId    String?
  cohort      Cohort?   @relation(fields: [cohortId], references: [id])
}

enum Tier { FREE INDIVIDUAL PRO TEAM_SEAT ENTERPRISE }

model Cohort {
  id          String    @id @default(cuid())
```

```

  name      String
  trainerId String
  trainer   User    @relation("TrainerCohorts", fields: [trainerId], references: [id])
  users     User[]
  startDate DateTime
  endDate   DateTime
}

```

```

model Scenario {
  id      String  @id
  slug    String  @unique
  title   String
  domain  String
  level   Level
  iterations Int
  definition Json    // full scenario config
  eventDeck Json
  authorId String?
  public   Boolean  @default(false)
}

```

```

enum Level { TUTORIAL BEGINNER INTERMEDIATE ADVANCED EXPERT }

```

```

model Game {
  id      String  @id @default(cuid())
  scenarioId String
  scenario Scenario @relation(fields: [scenarioId], references: [id])
  userId   String
  user     User    @relation(fields: [userId], references: [id])
  mode     Mode
  state    Json    // full serialized game state
  seed     String  // for deterministic replay
  status   Status  @default(IN_PROGRESS)
  iterations Iteration[]
  retro    Retrospective?
  createdAt DateTime @default(now())
  completedAt DateTime?
}

```

```

enum Mode { SOLO COOP TOURNAMENT CLASSROOM INTERVIEW }

```

```

enum Status { IN_PROGRESS COMPLETED ABANDONED }

```

```

model Iteration {
  id      String  @id @default(cuid())
  gameId   String
  game     Game    @relation(fields: [gameId], references: [id])
  number   Int
  backlogOrder Json // ordered list of PBI IDs + release card position
  sprintGoal String?
  events   Json    // events drawn this iteration
  decisions Json    // player responses to events
  outcome  Json    // releases, happiness deltas, score changes
  startedAt DateTime
  completedAt DateTime?
}

```

```

model Customer {
  // Customer data is embedded in Game.state JSON, not a separate table.
  // Rationale: customers are ephemeral per game; heavy joins would harm perf.
}

```

```

model Retrospective {
  id      String  @id @default(cuid())
  gameId   String  @unique

```



```

game      Game      @relation(fields: [gameId], references: [id])
headline  String
strengths Json
growthEdges Json
alternateHistory Json
techniqueUnlocked String?
recommendedNext String?
aiModel   String
generatedAt DateTime @default(now())
}

model UserProgress {
  userId    String    @id
  user      User      @relation(fields: [userId], references: [id])
  scenariosCompleted Int @default(0)
  techniquesUnlocked Json
  badges    Json
  elo       Int       @default(1000)
}

model EventLog {
  id        String    @id @default(cuid())
  gameId    String
  type      String    // decision | event | outcome | retro
  payload   Json
  timestamp DateTime @default(now())
  @@index([gameId, timestamp])
}

```

13. Feature Specifications (MVP)

F-01 · Onboarding & tutorial (3-iteration mini-game)

User story: As a new user, I want a 15-minute tutorial that teaches me the core mechanics so I can immediately play a real scenario.

AC-1 Signed-up user lands on tutorial automatically; skippable for returning users.

AC-2 Tutorial uses a 3-iteration SaaS scenario with 3 personas and a preset event deck.

AC-3 First iteration walks through drag-order, release card placement, and commit.

AC-4 Iter 2 introduces customer signals (inbox message).

AC-5 Iter 3 introduces an event with choice; tutorial ends with mini-retro.

AC-6 Completion unlocks first real scenario + writes `tutorial\_completed` to PostHog.

F-02 · Iteration planning board

User story: As a player, I want to drag items into an ordered iteration backlog and place a release card so I can commit to a plan.

AC-1 Left panel: full product backlog (scrollable, filterable by customer).

AC-2 Right panel: empty iteration backlog (drag target).

AC-3 Top: current Sprint Goal (editable text field, required before commit).

AC-4 Bottom: "Commit Sprint" button (disabled until Sprint Goal + 1 item placed).

AC-5 Drag-reorder with dnd-kit; accessible with keyboard (arrow keys + space).

AC-6 Release card is a draggable card (1 pt); players insert anywhere in ordered list.

AC-7 Effort total vs. expected velocity shown as a progress bar, not a number (zero-friction).

F-03 · Event engine + response modal

User story: As a player, when an event fires mid-sprint, I want to choose a response and see the consequence.

AC-1 Events fire between iterations based on weighted deck + behaviour modifiers.

AC-2 Modal shows event narrative, 2–3 choices, and visible consequence preview.

AC-3 Choice locks in; hidden consequences (tech debt, trust shifts) recorded silently.

AC-4 Events logged to EventLog table for retrospective analysis.

F-04 · Customer inbox (signals)

User story: As a player, I want to see customer messages so I can understand who's engaged and what they need.

AC-1 Inbox widget shows 3–10 messages per iteration, variable by customer archetype.

AC-2 Messages include: feature requests, bug reports, compliments, churn warnings, silent signals.

AC-3 Clicking a customer avatar shows their engagement history (graph of state transitions).

AC-4 Unread badge; messages marked read on click (tracked for analytics).

F-05 · Sprint Review (post-iteration recap)

User story: As a player, I want a clear recap of what happened each iteration so I can learn.

AC-1 Animated release moment: released items fly out, revenue number counts up.

AC-2 Customer reaction section: which customers changed state, why.

AC-3 Team health indicator: morale, velocity trend.

AC-4 Next iteration button starts the next planning phase.

F-06 · AI-generated retrospective (end of game)

User story: As a player, I want a personalised retrospective after each game so I know exactly what to improve.

AC-1 End-game triggers Claude Opus call with full game log.

AC-2 Loading screen shows game stats during generation (~20s wait).

AC-3 Retrospective displays: headline, 3 strengths, 3 growth edges, 1 alternate-history replay.

AC-4 Technique unlocked is added to UserProgress.

AC-5 "Recommended next scenario" CTA.

AC-6 Shareable link (public URL) for retrospective card (LinkedIn-friendly).

F-07 · User progress & scenario library

User story: As a player, I want to see my progress and pick my next challenge.

AC-1 Home screen: recommended scenario, recent retrospectives, techniques unlocked.

AC-2 Library: all scenarios, locked or unlocked by tier + prerequisites.

AC-3 Each scenario card shows: difficulty, est. time, what it teaches, completion status.

F-08 · Auth & payment

User story: As a user, I want to sign up and upgrade to unlock more scenarios.

AC-1 Email + Google auth via Clerk.

AC-2 Free tier: tutorial + scenario 01 + 02 (2 real scenarios).

AC-3 \$19/mo Individual unlocks all scenarios.

AC-4 Stripe Checkout integration. Cancel anytime.

14. UI / UX Design Principles

Five zero-friction principles

1. **No numbers on screen unless essential.** Use progress bars, sentiment icons, message tone — not integers.
2. **One primary action per screen.** Planning = drag + commit. Review = read + continue. Retro = absorb + next.
3. **Animations carry meaning.** A customer becoming an advocate gets a real celebration moment. A churn gets a quiet dimming, not a red banner.
4. **Progressive disclosure.** Hide advanced mechanics (tech debt meter, stakeholder trust) until the player's behaviour unlocks them.
5. **No manual saves.** Every decision auto-persists. Game is always resumable from any device.

Key screens (MVP)

1. **Onboarding** — 3 screens: welcome, tutorial start, first drag
2. **Home/Library** — scenarios + progress + recent retros
3. **Planning board** — the drag-drop canvas (most used screen)
4. **Customer inbox** — message list + customer detail panel
5. **Event modal** — event narrative + choices
6. **Sprint Review** — animated recap
7. **End-game retrospective** — AI-generated report
8. **Account/Billing** — settings + Stripe portal

Design system

shadcn/ui as base. Custom components for card drag, release moment animation, customer avatar grid.
Typography: Inter. Colour palette: warm neutrals + deep teal accent + single "victory" gold for releases.

Accessibility (non-negotiable for MVP)

- Full keyboard navigation for drag-drop (dnd-kit built-in)
- Screen reader-friendly state changes
- WCAG AA contrast throughout
- Reduced-motion mode for animations

15. Analytics & Instrumentation

PostHog event schema (MVP)

Event	Key properties
game_started	scenario_id, mode, user_tier
iteration_planned	game_id, iter_number, release_card_position, items_ordered
iteration_committed	game_id, iter_number, sprint_goal_length, sprint_goal_clarity_score
event_fired	game_id, iter_number, event_type
event_responded	game_id, event_id, choice_id, time_to_decide_ms
release_occurred	game_id, iter_number, products_released, revenue
customer_state_transition	game_id, customer_id, from_state, to_state, trigger
retro_generated	game_id, technique_unlocked, generation_cost_usd
retro_shared	game_id, channel
subscription_started	tier, mrr

Key dashboards

- **Funnel:** visit → signup → tutorial complete → scenario 1 complete → paid subscription
 - **Engagement:** games per user per week, time-to-first-decision, avg session length
 - **Learning outcomes:** techniques unlocked per player, retro sharing rate
 - **Business:** MRR, churn, CAC by channel, LTV by tier
 - **AI cost:** cost per retro, cache hit rate, cost per game by tier
-

16. Security, Privacy & Compliance

Auth & access control

- Clerk for auth (email + Google + LinkedIn). MFA optional free; required for enterprise tier.
- Row-level security via Prisma middleware — users only see their own games.
- Trainer/cohort roles: CSTs can view games of users in their cohort.

Data handling

- PII (email, name) stored in Postgres with encryption at rest (Supabase default).
- Game data (no PII beyond user_id) retained indefinitely unless user requests deletion.
- AI prompts never include PII — only game state IDs and generated narrative.
- Stripe handles all payment data (PCI offloaded).

Privacy compliance

- GDPR: right to export (Game + Retrospective JSON), right to delete (hard delete with cascade).
- CCPA: Do Not Sell toggle in account settings.
- Canadian PIPEDA: relevant for Mike's home jurisdiction; handled by above controls.
- Cookie banner with analytics opt-out.

AI safety

- Retrospective prompts include explicit system message: "Feedback must be constructive, specific, never shaming."
- User-facing AI outputs labelled "Generated by Claude".

- Anthropic's Prompt Shield + usage policies enforced.

17. Pricing & Business Model

Tiers

Tier	Price	Features	Target
Free	\$0	Tutorial + 2 scenarios · 1 retro/mo · Solo only	Acquisition
Individual	\$19/mo or \$149/yr	All scenarios · Unlimited retros · Progress tracking	Aspiring / mid-career PMs
Pro	\$49/mo or \$399/yr	Individual + Tournament + Multiplayer co-op · Interview mode · Paste-your-backlog	Working PMs · founders · job seekers
Team	\$199/mo (10 seats)	Pro × 10 + Private cohort · Admin dashboard · Team leaderboard	Scrum teams training internally
CST Partner	Rev share 70/30	Branded cohort URL · Trainer dashboard · Co-branded certificates	Certified Scrum Trainers
Enterprise	\$15K+ /yr	Custom scenarios · SSO · DPA · On-site training kits · SLA	F500 L&D departments

Unit economics (Individual tier)

Monthly price	\$19
Stripe fees (2.9% + 30¢)	−\$0.85
AI costs (~10 retros/mo × \$0.12)	−\$1.20
Hosting/infra (per active user, amortised)	−\$0.50
Gross margin / user / month	\$16.45 (87%)
Assumed avg churn	7%/mo
Avg customer lifetime	14 months
LTV (Individual)	\$230
Target CAC	< \$70 (3:1 LTV:CAC)

Conversion funnel assumptions

Visit → signup	10%
Signup → tutorial complete	60%
Tutorial → free scenario complete	50%

Free scenario complete → paid	8%
Overall visit → paid	0.24%

18. Go-to-Market Strategy

Phase 1 (Months 0–3): Wedge — PM Interview Simulator

- Launch one polished scenario: the "Interview Scenario" (30-min scored)
- Price: \$29 single-play or \$19/mo unlimited
- Distribution: Product Hunt launch, Hacker News Show HN, LinkedIn organic from Mike's network
- Content: 10 blog posts on "I practiced N PM scenarios. Here's what I learned." — long-tail SEO
- Target: 500 signups, 50 paying = ~\$1,000 MRR by month 3

Phase 2 (Months 3–6): CST Partnership Program

- Direct outreach to 50 top CSTs (Scrum Alliance directory)
- Offer: free branded cohort instance for their next workshop, 70/30 rev share
- Signal: "Zuzi Šochová uses PRAXIS for her CSPO cohorts" would unlock the channel
- Target: 5 signed CSTs, 500 student-users, +\$2,500 MRR

Phase 3 (Months 6–12): Enterprise pilots

- Target 20 mid-market L&D teams that have recently adopted Scrum
- Offer: 3-month pilot at \$5K; if successful, \$15K annual
- Sales channel: Mike's Questrade network + LinkedIn Sales Navigator
- Target: 2 signed enterprises = +\$2,500 MRR

Positioning & messaging

Audience	Hook
Aspiring PMs	"Leetcode, but for product judgment."
Working PMs	"Rehearse your next sprint before running it."
CSTs	"Your workshop, continued — as a digital cohort."
Enterprise L&D	"Scrum training with measurable outcomes, for under \$50 a seat."

Content engine

- Weekly blog post dissecting a famous PM decision (Slack pivot, Nokia fall, Figma merger)
- Monthly "PRAXIS leaderboard" — best decisions of the month, with commentary
- Shareable retro cards — every player's retro has LinkedIn share button; viral loop

- Podcast appearances: Lenny's, Marty Cagan's, This Is Product Management

19. Cost Projections & Financials

Development costs (MVP, 6 weeks)

Category	Cost	Notes
Mike's time (6 weeks × 30 hrs)	\$0 (sweat)	Founder opportunity cost
Claude Code subscription	\$100/mo × 2 mo = \$200	Accelerates build speed
Design work (1 scenario polish)	\$1,500	Contract designer for Sprint Review animations + illustrations
Scenario content writing	\$500	Scenario 01 narrative + events
Legal (terms, privacy)	\$500	One-time template
Incorporation & domain	\$300	CCPC + domain + SSL
Pre-launch total	\$3,000	

Monthly operating costs (at 500 users, ~50 paid)

Line item	Cost/mo
Vercel Pro	\$20
Supabase Pro (Postgres + storage)	\$25
Upstash Redis (free tier sufficient)	\$0
Clerk (Startup tier)	\$25
Stripe (transactions)	passthrough (already in margin)
PostHog (Cloud, under 1M events)	\$0
Sentry	\$26
Domain / misc	\$10
Claude API (500 users avg 5 retros/mo)	~\$300
Email (Resend/Postmark)	\$20
Marketing (content + ads)	\$500
Total monthly opex	~\$926

Year-1 P&L projection

Quarter	Paying users (end)	MRR (end)	Quarterly revenue	Quarterly opex	Net
---------	--------------------	-----------	-------------------	----------------	-----

Q1 (MVP launch)	20	\$400	\$800	\$4,000	−\$3,200
Q2 (CST partnerships)	120	\$2,800	\$4,800	\$5,500	−\$700
Q3 (growth)	350	\$8,000	\$16,000	\$9,000	+\$7,000
Q4 (enterprise pilots)	700	\$18,000	\$38,000	\$14,000	+\$24,000
Year 1 total	700	\$18,000	\$59,600	\$32,500	+\$27,100

ARR run-rate at end of year 1: \$216K. Break-even around month 8 assuming the above trajectory and no hiring.

Capital requirements

- **Bootstrap feasible.** Total cash burn year 1 ≈ \$11K (Q1+Q2 losses). Mike covers from savings or DentinFlow cashflow.
- **Optional raise:** \$250K pre-seed at \$2M cap for growth acceleration — hire 1 FT engineer by month 6.

20. Roadmap & Phasing

MVP — 6 weeks

Week	Deliverable
1	Repo setup · Auth · Prisma schema · 1 scenario hardcoded · Game engine core
2	Iteration planning board (drag-drop) · Commit + basic event firing
3	Customer inbox · Sprint Review screen · State transitions
4	Claude integration · Retrospective generator · Event deck complete for scenario 01
5	Tutorial · Billing (Stripe) · Polish animations · Accessibility pass
6	Soft launch: private beta to 20 CSPO cohort peers · Iterate · Public launch

Phase 2 — Months 3–6

- 9 more scenarios (03–10) · Scenario authoring tool for internal use
- CST partnership dashboard + private cohort URLs
- Multiplayer co-op (PO/SM/Dev) · WebSocket infrastructure
- Interview mode standalone product
- Public retrospective sharing · Referral program

Phase 3 — Months 6–12

- Tournament mode · Leaderboards · ELO system
- Paste-your-backlog (Linear/Jira import)
- Enterprise features: SSO, admin dashboard, cohort reporting

- Mobile-responsive / PWA
- Year campaign mode (24 iter) · Portfolio mode

Phase 4 — Year 2

- Community-contributed scenarios
- Stakeholder roleplay via AI (Sonnet)
- Integrations: Slack, Microsoft Teams, Linear, Jira
- Mentor marketplace (senior PMs charge for 1:1 retro reviews)
- Credentialing path · Industry-recognised certifications

21. Success Metrics & KPIs

North Star metric

Weekly Active Deciders (WAD): users who made at least 5 consequential decisions in the past 7 days. Captures engagement in the thing that matters (practice), not vanity metrics (logins).

Product health metrics

- **Tutorial completion rate:** > 60% of signups
- **Scenario 01 completion rate:** > 50% of tutorial-completers
- **Games per active user per week:** > 2.0
- **Retrospective share rate:** > 10% of retros shared to LinkedIn
- **Time to first commit (from signup):** < 15 min

Business metrics

- **Paid conversion (free → Individual):** > 5% within 30 days
- **Monthly churn:** < 7%
- **CAC payback:** < 4 months
- **NPS:** > 40 at 6 months (PM training benchmark ~30)
- **LTV:CAC ratio:** > 3:1

Learning outcome metrics (unique to PRAXIS)

- **Techniques unlocked per user:** median > 5 within 90 days
- **Decision quality improvement:** ratio of "good" decisions (per AI mentor) per game, tracked over a player's history

- **Certification pass rate:** for paired CSPO candidates, pass rate > baseline

22. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
AI retrospectives are generic / low-quality	Medium	High	Extensive prompt engineering · user testing · iteration · structured output validation · Opus 4.7 for depth
CSTs resist because it competes with them	Medium	High	Position as augmentation not replacement · generous rev share · co-branded certificates
Game becomes "too gamey" — PMs don't take seriously	Medium	High	Professional tone · real scenarios · named industry mentors · evidence-backed content
AI costs scale worse than expected	Medium	Medium	Aggressive prompt caching · Haiku for in-game · batch retros · tier-gate high-cost features
Chicken-and-egg for multiplayer	High	Medium	Solo mode is fully viable on its own · multiplayer rolls out when cohorts exist
GoPractice / Reforge launches a competing simulator	Low	High	Speed · own the Scrum/PO niche · build moat via scenario library + AI quality
Anthropic pricing increases significantly	Low	Medium	Multi-provider abstraction (OpenAI, Google fallback) · usage caps
Mike burns out juggling DentinFlow + PRAXIS	High	High	Pick ONE product for year 1 · park the other · be explicit with self about priority
Scenario authoring is bottleneck	High	Medium	AI-assisted scenario generation by Month 6 · community contribution Year 2
Accessibility gaps hurt enterprise sales	Medium	Medium	WCAG AA from MVP · third-party audit before enterprise launch

Top risk: founder overcommitment. Mike has DentinFlow already in flight. PRAXIS is a separate product requiring focus. Recommend making an explicit choice before starting: PRAXIS full-time, or DentinFlow full-time with PRAXIS in weekend mode only. Not both concurrently as primary.

23. Team & Resourcing

Founding (year 1)

- **Mike Hamata** — CEO, Product, Engineering, GTM. Full-time from month 0 if PRAXIS becomes the chosen focus.
- **Claude Code + Claude Opus 4.7** — primary "force multiplier" development partner. Estimated 3–5× throughput vs. solo.

- **Contractors** — designer (10 hrs, \$1,500), legal (one-time).

First hires (months 6–12)

Role	When	Why
Full-stack engineer	Month 6 if \$10K+ MRR	Scenario authoring tooling, enterprise features
Content / scenario designer	Month 9	Build out library to 40 scenarios
Customer success / CST liaison	Month 12	Manage 20+ CST partnerships

Advisory board targets

- One CST (Zuzi Šochová ideal — relationship already open)
- One senior PM (Shreyas Doshi, Lenny Rachitsky-tier)
- One educator from Stanford GSB or MIT Sloan for academic credibility
- One YC founder for GTM advice

24. Next Steps

Immediate (next 7 days)

1. Mike decides: PRAXIS full-time OR weekend side project. Write it down.
2. Lock working name (PRAXIS or alternative). Register domain + trademark check.
3. Set up GitHub repo: `mikehamata/praxis` . MIT license. Public repo for community credibility.
4. Brief Zuzi Šochová on the idea. Ask for a 30-min call. Offer free pilot for her next cohort.
5. Claude Code session: scaffold Next.js + Prisma + Clerk + Stripe · seed scenario 01

Weeks 1–6 (MVP build)

1. Follow the 6-week roadmap in Section 20.
2. Weekly demo to 3 chosen beta users (Mike's CSPO cohort members).
3. Instrument PostHog events from day 1.
4. Write blog posts (2 per week) as build-in-public marketing.

Week 7 (launch)

1. Product Hunt launch (Tuesday, optimal day).
2. Show HN post.
3. LinkedIn long-form from Mike's account: "Why I built PRAXIS."
4. Email all CSPO alumni + Scrum Alliance contacts.
5. Offer lifetime 50% discount for first 100 paying customers.

Decision gates

Gate	Target	Action if missed
End of Month 1	100 signups	Pivot GTM messaging
End of Month 3	\$1K MRR	Question PMF; interview 20 churned users
End of Month 6	\$5K MRR + 3 CST partners	Slow down hiring; extend runway
End of Month 12	\$15K MRR	Decide on raise vs. continued bootstrap

Success criteria for continuing after Month 6: if MRR > \$5K AND at least 2 CST partnerships signed AND weekly active deciders > 100, *commit to year 2 and consider raise.*

Document status: Draft v1.0 · Pending stakeholder review · 2026-04-18

Review cycle: This PRD should be revisited after: (a) initial user research with 5 CSPO cohort members, (b) first Claude Code build week, (c) first 20 paying users. Update version number with each significant revision.

Co-author acknowledgement: This PRD was co-authored by Mike Hamata and Claude Opus 4.7. All decisions are Mike's; Claude contributed research synthesis, market analysis, technical architecture proposals, and document structuring.